

SE-Blackboard: A Shared-State Architecture for Multi-Agent Software Engineering Pipelines

ERXI LIU¹, QIANG ZHU², XINRU DONG^{3,2}

¹Tandon School of Engineering, New York University, New York, United States (e-mail: el4435@nyu.edu)

²The National Institute of Metrology of China, Beijing, China (e-mail: zhuqiang@nim.ac.cn)

³School of Mechanical and Electrical Engineering, Beijing Institute of Graphic Communication, Beijing, China (e-mail: 18339301370@163.com)

Corresponding author: Qiang Zhu (e-mail: zhuqiang@nim.ac.cn).

ABSTRACT Multi-agent large language model (LLM) systems are increasingly used for software engineering tasks, yet the role of **communication architecture** in these systems remains poorly understood. Most existing approaches assume message passing, where agents exchange sequential natural-language summaries, without examining the information loss this assumption may introduce. This paper makes a methodological and diagnostic contribution to that question. We present **SE-Blackboard**, a modular framework that places message passing, a shared-state blackboard, and a hybrid design within an otherwise identical multi-agent pipeline, and we introduce the **Information Fidelity Score (IFS)**, a stage-level metric that measures how well key technical entities from the original issue survive successive agent handoffs. On 50 SWE-bench Lite issues, the blackboard configuration improves Coder-stage IFS by 62% over message passing (0.586 vs. 0.362) and correct file targeting by 27.5 percentage points (82.6% vs. 55.1%). Once the correct file is identified, however, all configurations converge to a $\sim 22\%$ conditional resolve rate, revealing a downstream bottleneck in LLM diff generation. Building on these decompositions, we present an **information-flow bottleneck model** that predicts how upstream gains attenuate at the patch-generation stage; the model accounts for the modest end-to-end resolve-rate improvement observed (16% vs. 12%). These findings identify communication architecture as a measurable, separable design dimension and supply tools—IFS and the bottleneck model—for diagnosing where future improvements in multi-agent SE pipelines should be targeted.

INDEX TERMS multi-agent systems, large language models, software engineering, blackboard architecture, shared state, communication architecture, information fidelity, SWE-bench

I. INTRODUCTION

THE rapid advancement of large language models (LLMs) [1] has catalyzed the emergence of multi-agent systems for software engineering (SE). Systems such as MetaGPT [2], ChatDev [3], SWE-Agent [4], and AgentCoder [5] demonstrate that multiple AI agents can collaboratively solve complex tasks, including code generation [6], bug fixing, and code review, representing a shift from monolithic tools toward modular, role-based architectures.

However, current multi-agent SE systems share a common architectural assumption: **message-passing communication**. Agents communicate by forwarding natural-language messages along a predefined chain; the output of one agent becomes the input to the next. This approach has a fundamental limitation: information fidelity degrades at each handoff. Critical technical information, such as function signatures, parameter types, and error stack traces, may be paraphrased,

simplified, or omitted. We term this progressive degradation of information *knowledge drift*: the cumulative loss or distortion of technical entities, identifiers, and references that occurs as task-relevant content is paraphrased through successive agent handoffs, leading downstream agents to operate on a representation that no longer fully reflects the original problem specification. Knowledge drift has also been identified as a primary failure cause in multi-agent LLM systems [7].

Knowledge drift is particularly critical in software engineering contexts. While paraphrasing in general reasoning tasks may preserve semantic equivalence, SE tasks require exact technical precision. A single character difference in a function name or file path can determine whether a patch applies correctly, making SE an ideal domain for studying information preservation in multi-agent architectures.

Existing approaches to communication quality have focused on post-hoc remediation. RTADev [8] proposes inten-

tion alignment to verify that agents' interpretations match the original intent, but only detects information loss after it has occurred. The Blackboard architecture, on the other hand, offers an alternative. It is a classical AI pattern featuring a shared workspace accessible to all agents [9]. Recent work has demonstrated Blackboard's effectiveness for LLM-based reasoning [10] and data science [11] tasks, but it has never been applied to software engineering.

This paper makes the following contributions:

- 1) **SE-Blackboard Framework:** The application of the Blackboard shared-state architecture to multi-agent software engineering, which includes three pluggable communication modes (Message-Passing, Blackboard, and Hybrid) within identical agent pipelines.
- 2) **Information Fidelity Score (IFS):** A new metric that measures the preservation of key technical entities across pipeline stages, enabling measurement of information preservation independent of downstream task success.
- 3) **Controlled Empirical Evaluation:** Experiments on 50 SWE-bench Lite issues demonstrate that BB improves Coder-stage IFS by 62% (0.586 vs. 0.362, $p = 0.058$) and resolve rate by 4 percentage points (16% vs. 12%), with patch formatting identified as the primary bottleneck.
- 4) **Open-Source Framework:** A modular framework with pluggable communication architectures, structured state schemas, and comprehensive logging for future research.

II. RELATED WORK

A. MULTI-AGENT SYSTEMS FOR SOFTWARE ENGINEERING

Recent studies have introduced a range of multi-agent software engineering (SE) systems that differ in architectural design and interaction style, which typically rely on the code-generation strengths of foundation models such as Codex [6] and Code Llama [12]. Role-oriented frameworks—such as MetaGPT [2], ChatDev [3], and AutoGen [13]—assign specialized responsibilities to individual agents and organize their collaboration through predefined workflows or structured dialogue. In contrast, tool-augmented approaches such as SWE-Agent [4] and AutoCodeRover [14] focus on enhancing a single agent with repository navigation and execution tools, achieving strong performance without explicit multi-agent coordination.

For automated bug fixing, AgentCoder [5] and CodeR [15] use multi-agent pipelines that incorporate iterative feedback from test execution, whereas Agentless [16] shows that carefully designed prompting strategies can attain comparable results without deploying multiple agents. More recently, MAGIS [17] proposes an LLM-based multi-agent framework specifically for GitHub issue resolution, organized around role-specialized agents that exchange information through structured handoffs; its design goals are closely aligned with ours, though it does not isolate the communication architec-

ture as an independent variable. This comparison suggests that interaction design may be as influential as agent multiplicity itself.

Recent advances also extend agent capability through tool integration: SWE-Agent [4] formalizes an agent–computer interface that exposes repository navigation and file editing as discrete tools, AutoCodeRover [14] couples spectrum-based code search with iterative LLM reasoning, and Agentless [16] achieves comparable results using only carefully structured prompts. These developments share a common emphasis on what each agent can *do* (search, read, execute, retry) rather than how multiple agents *exchange* information among themselves.

Despite their differences, these systems largely rely on message passing as the primary mechanism for inter-agent communication. Although some approaches use shared artifacts, the communication architecture is seldom examined as an independent design dimension. Our work seeks to fill this gap.

B. COMMUNICATION ARCHITECTURES FOR LLM AGENTS

He et al. [18] provide a survey of LLM-based multi-agent systems for software engineering, outlining several communication patterns, including direct messaging, shared memory, and blackboard-style coordination. They observe, however, that many existing systems still rely on relatively simple sequential exchanges. Cemri et al. [7] examine common failure modes in multi-agent LLM systems and identify knowledge drift as a central issue, showing that drift becomes more evident as the length of agent interaction chains increases.

Alongside research on communication architectures, recent studies focus on strengthening the reasoning abilities of individual agents in multi-agent settings. Chain-of-Thought prompting [19] supports stepwise reasoning, while ReAct [20] integrates reasoning with tool use, grounding agent decisions in environmental feedback. Reflexion [21] introduces a verbal reinforcement-learning framework that enables agents to iteratively refine their behavior by reflecting on prior mistakes. Park et al. [22] further demonstrate that generative agents with memory and reflective mechanisms can display coherent collaborative behavior, providing a firm basis for more advanced coordination strategies.

More recently, RTADev [8] introduces intention alignment as a reactive mechanism that mitigates information loss after it occurs. By contrast, the Blackboard architecture adopted in our work takes a proactive approach, ensuring that all agents retain direct access to source information regardless of intermediate processing steps.

C. BLACKBOARD ARCHITECTURE

The Blackboard architecture, first introduced by Nii [9], describes a framework in which multiple knowledge sources coordinate through a shared workspace, which was initially applied to speech understanding and planning systems [23]. Recently, this framework has attracted further interest in the context of LLM-based multi-agent coordination. Wang et

al. [10] show that communication through a shared state can reduce information loss compared with message passing in reasoning tasks, and the benefits become more evident as task complexity and the length of agent interaction chains increase. Salemi et al. [11] explore this pattern in data science workflows and report improved coherence when agents can directly observe each other's outputs.

To our knowledge, this work brings the approach to software engineering for the first time. Software engineering tasks introduce additional challenges: the system state must capture heterogeneous artifacts such as issue descriptions, code context, patches, and test results while maintaining clear ownership semantics. Moreover, code-related information requires a much higher degree of precision than typical natural language reasoning.

III. SYSTEM DESIGN

SE-Blackboard is a modular framework that aims to study how communication architecture affects performance on software engineering tasks. As shown in Figure 1, the system consists of four main components: a structured state schema, three interchangeable communication modes, four specialized agents, and a pipeline-based topology.

A. SE STATE SCHEMA

The framework uses a domain-specific state schema implemented as Pydantic models to represent the key artifacts produced during the bug-fixing pipeline, including **IssueInfo** (the original issue description, repository, and base commit), **Analysis** (the Planner's root-cause analysis and proposed fix strategy), **Patch** (the Coder's generated unified diff), **Review** (the Reviewer's accept/reject decision along with feedback), and **TestResult** (the Tester's pass/fail evaluation).

Formally, the shared state is defined as a tuple:

$$\mathbf{S} = (I, A, P, R, T)$$

where I denotes IssueInfo, A denotes Analysis, P denotes Patch, R denotes Review, and T denotes TestResult.

A key design principle is **field ownership**. Each agent is only allowed to write to its designated state fields and has read access to upstream fields. Formally, the write-permission mapping $W : \mathcal{K} \rightarrow 2^{\{I, A, P, R, T\}}$ assigns each agent role $k \in \mathcal{K} = \{\text{Planner}, \text{Coder}, \text{Reviewer}, \text{Tester}\}$ the subset of state fields that role is permitted to modify. It is defined as follows:

$$\begin{aligned} W(\text{Planner}) &= \{A\}, & W(\text{Coder}) &= \{P\}, \\ W(\text{Reviewer}) &= \{R\}, & W(\text{Tester}) &= \{T\} \end{aligned}$$

The IssueInfo field I is excluded from every $W(k)$, making it read-only for all agents. This ownership model prevents agents from inadvertently overwriting each other's contributions and ensures a clear audit trail. The combination of a structured shared state with role-bound write permissions is consistent with recent cognitive-architecture treatments

of language agents [24], which advocate explicit separation between working memory, reasoning state, and action production—a separation our $\mathbf{S} = (I, A, P, R, T)$ schema realizes at the granularity of pipeline stages.

Crucially, the original issue description (IssueInfo) is always preserved in the shared state and remains directly accessible to all agents. This design mitigates knowledge drift: downstream agents, such as the Coder, can reference the original issue text directly rather than relying solely on the Planner's summarization.

B. COMMUNICATION MODES

The framework supports three communication modes (Figure 1, bottom):

Message-Passing (MP): Each agent receives only the output of its immediate predecessor as natural language input and does not share structured state. This configuration reflects many existing multi-agent SE systems.

Blackboard (BB): All agents read from and write to a shared state instance. Each agent is provided with a role-specific view through `get_state_for_agent(role)`, which includes all upstream fields together with the original issue description. The Coder receives both the Planner's structured analysis *and* the original issue text, enabling cross-referencing.

Hybrid: Agents have access to both message-passing context and the shared blackboard state, combining the two information channels.

We formalize the modes using a context function $C(k)$, which specifies the information available to the k -th agent at execution time. Let $\text{out}(k-1)$ denote the natural language output of the preceding agent, S_j denote the state field written by agent j , and I denote the original issue description.

Message-Passing:

$$C^{\text{MP}}(k) = \{\text{out}(k-1)\}$$

Blackboard:

$$C^{\text{BB}}(k) = \{S_j \mid j < k\} \cup \{I\}$$

Hybrid:

$$C^{\text{Hy}}(k) = \{\text{out}(k-1)\} \cup \{S_j \mid j < k\} \cup \{I\}$$

The key architectural difference is that $I \in C^{\text{BB}}(k)$ and $I \in C^{\text{Hy}}(k)$ but $I \notin C^{\text{MP}}(k)$ for downstream agents ($k \geq 2$). In BB mode, the Coder always has direct access to the original issue text. On the other hand, in MP mode, the Coder obtains information about the issue only through the Planner's summarized output `out(Planner)`. This distinction is the formal basis for the differences in information fidelity analyzed in Section V-B.

All three modes share identical agent implementations and prompt templates, with only the context formatting varying by communication mode, ensuring that observed differences are attributable to the communication architecture.

SE-Blackboard: System Architecture

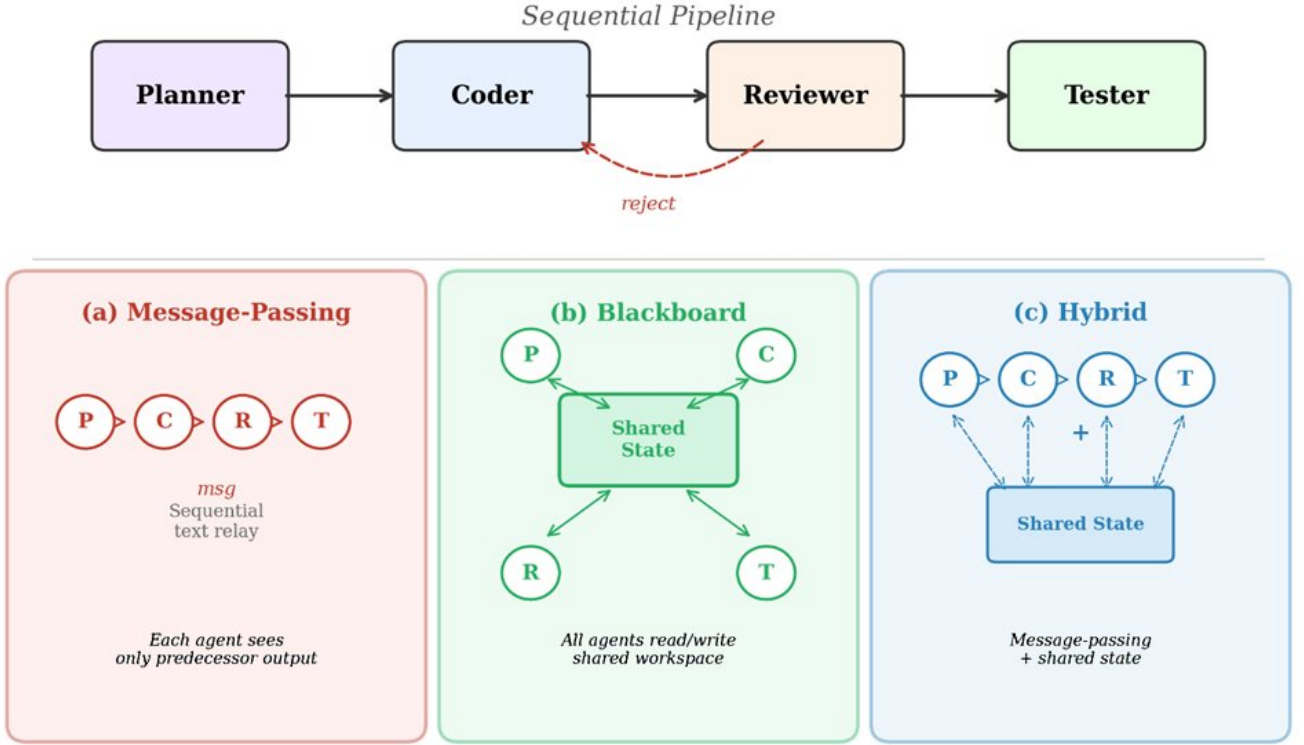


FIGURE 1. SE-Blackboard system architecture. The framework consists of a structured state schema, three interchangeable communication modes (Message-Passing, Blackboard, Hybrid), four specialized agents, and a sequential pipeline topology.

C. AGENT DESIGN

The pipeline consists of four agents: **Planner**, **Coder**, **Reviewer**, and **Tester**. The Planner analyzes the issue to produce a root cause analysis and a fix strategy. The Coder generates a unified diff patch from the analysis and repository source code. The Reviewer evaluates the patch and provides accept/reject feedback, using a feedback-driven iteration mechanism similar to Reflexion [21]. Finally, the Tester evaluates the resulting patch using the SWE-bench Docker harness. Each agent uses prompt variants tailored to the communication mode.

D. PIPELINE TOPOLOGY

The framework follows a sequential pipeline with a rejection loop from the Reviewer back to the Coder (up to 3 iterations). Once the patch is accepted or the maximum number of iterations is reached, the Tester evaluates the resulting patch. The Tester always executes regardless of the Reviewer's decision, ensuring that each run produces a test result.

Formally, let $\mathcal{P}$, $\mathcal{C}$, $\mathcal{R}$, and $\mathcal{T}$ denote the Planner, Coder, Reviewer, and Tester agents, respectively, and let N denote the maximum number of Coder–Reviewer iterations. The pipeline output is:

$$\text{output} = \mathcal{T} \circ \left(\bigotimes_{n=1}^N \mathcal{R} \circ \mathcal{C} \right) \circ \mathcal{P}(I)$$

where the inner loop terminates early if $\mathcal{R}$ returns an accept decision at iteration $n < N$.

IV. EXPERIMENTAL SETUP

A. BENCHMARK

We evaluate on SWE-bench Lite [25], which is a curated subset of 300 real-world GitHub issues from 12 popular Python repositories. Each issue includes a natural language problem description, a base commit, and test cases that the fix must pass (FAIL_TO_PASS) while not breaking existing functionality (PASS_TO_PASS).

We select a stratified sample of 50 issues to balance evaluation cost, repository diversity, and difficulty coverage while remaining comparable to prior small-scale SWE-bench Lite studies. Issues were initially sampled in equal strata based on repository metadata to avoid concentration on any single project, then reclassified post-hoc using gold patch complexity as an objective difficulty proxy: Easy (single-file, <20 lines): 22 issues, predominantly Django; Medium (single-file, 20–50 lines or multi-file with small changes): 5 issues; Hard (multi-file, >50 lines, or complex reasoning): 23 issues.

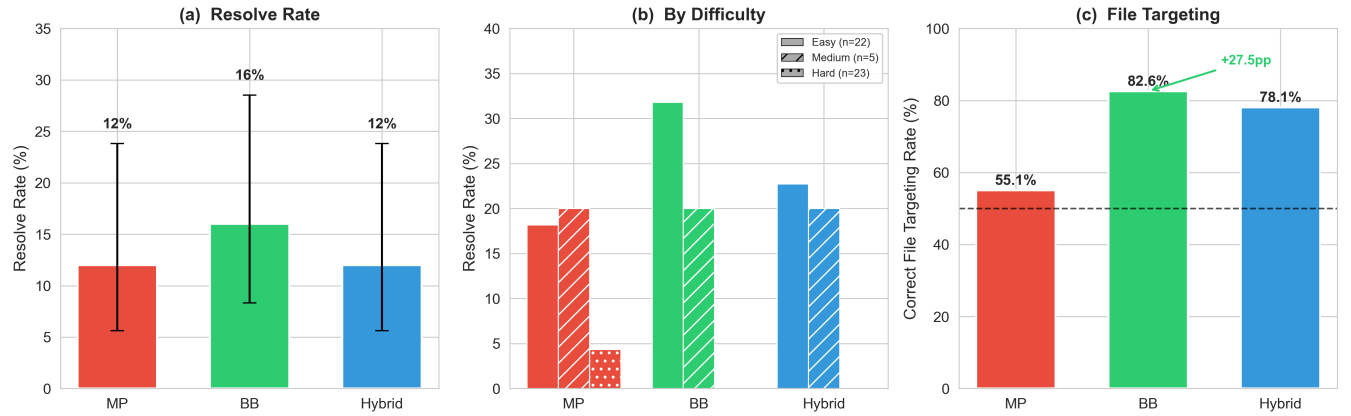


FIGURE 2. Main experimental results. (a) Overall resolve rate by communication mode. (b) Resolve rate by difficulty level. (c) Correct file targeting rate among non-empty patches.

The final sample covers 8 of the 12 repositories in SWE-bench Lite, with Django overrepresented (22/50) because its comprehensive test coverage makes a larger fraction of issues passable under the FAIL_TO_PASS/PASS_TO_PASS evaluation protocol; we discuss the implications of this concentration for external validity in Section VII.

B. CONFIGURATIONS

We evaluate three configurations that combine the Sequential Pipeline with different communication modes:

TABLE 1. Experimental configurations for the three communication modes.

Configuration	Communication	Label
A-Seq(MP)	Message-Passing	MP
B-Seq(BB)	Blackboard	BB
C-Seq(Hy)	Hybrid	Hybrid

All configurations use identical hyperparameters: Claude Sonnet 4.5 (claude-sonnet-4-5-20250929), temperature 0, a maximum of 3 Coder-Reviewer iterations, and function-level code context retrieved via git checkout. The single LLM with deterministic generation ensures that observed differences are attributable to the communication architecture. Each issue is evaluated once per configuration, yielding 150 total runs.

C. EVALUATION METRICS

Resolve Rate. An issue is resolved if the generated patch causes all FAIL_TO_PASS tests to pass while maintaining PASS_TO_PASS tests. Evaluation is performed using the official SWE-bench Docker harness.

Information Fidelity Score (IFS). We extract key technical entities from the original issue (function names, class names, file paths, error types) using rule-based patterns, then measure the proportion of these entities that appear in each agent’s output:

$$\text{IFS}_s = \frac{|\text{entities in agent } s\text{'s output}|}{|\text{entities in issue description}|}$$

In MP mode, IFS is computed only at the Coder stage; in BB and Hybrid modes, it is computed at all four stages. The rule-based extraction may undercount semantically equivalent references; Section VII discusses this limitation.

Patch Quality Metrics. To localize the performance bottleneck, we analyze the patch generation rate (non-empty output), the correct-file targeting rate (the proportion of non-empty patches that modify the same file(s) as the gold patch), and the conditional resolve rate (among correct-file patches that pass evaluation).

Efficiency Metrics. We measure token consumption, latency, and iteration count per issue.

Relationship to prior multi-agent evaluation criteria. IFS differs in scope from common multi-agent evaluation criteria, which include end-to-end task success (e.g., resolve rate), role-consistency metrics, and dialogue-quality scores. Task-success metrics summarize the whole pipeline into a single binary outcome and therefore conflate communication quality with downstream generation capability; role-consistency and dialogue-quality metrics target generic conversational behavior and do not test whether domain-specific technical entities (function names, file paths, error types) survive each handoff. IFS instead measures stage-level preservation of those technical entities and is therefore designed to isolate the contribution of the communication layer from confounds such as patch-formatting reliability. We use IFS alongside, not in place of, resolve rate.

D. STATISTICAL METHODS

Given the paired design (each issue under all three configurations), we use McNemar’s exact test [26] for resolve rate comparisons, Wilcoxon signed-rank test [27] for continuous measurements (tokens, latency, IFS), Cohen’s h for effect size, and odds ratio with 95% CI. All tests use $\alpha = 0.05$. With $N = 50$ and expected resolve rates of 12–16%, statistical power to detect resolve-rate differences is limited; therefore, we interpret the results in conjunction with IFS and failure analysis.

V. RESULTS

A. RESOLVE RATE

Table 2 presents the main experimental results across three communication architectures.

TABLE 2. Main experimental results on SWE-bench Lite ($N = 50$, Sequential pipeline).

Comm.	Res.	Rate	Tokens	Latency
MP	6/50	12.0%	25,316	106s
BB	8/50	16.0%	54,849	171s
Hybrid	6/50	12.0%	43,278	128s

The Blackboard configuration resolves 8/50 issues (16.0%), compared to 6/50 (12.0%) for both MP and Hybrid, corresponding to a 4 percentage-point absolute improvement and 33% relative improvement (Figure 2a). However, McNemar's exact test yields $p = 0.625$ (see Table 10 in Section V-H) based on only 4 discordant pairs ($b = 1, c = 3$), indicating insufficient statistical power. Cohen's $h = 0.116$ confirms a negligible effect size, and the odds ratio of 1.40 (95% CI [0.45, 4.37]) does not exclude the null hypothesis.

1) Difficulty Breakdown

The advantage of the Blackboard configuration is concentrated on easy-difficulty issues (Table 3, Figure 2b). Among 22 easy issues, BB resolves 7 (31.8%) compared with MP's 4 (18.2%). In the medium category (5 issues), all configurations resolve exactly 1 (20.0%). Among 23 hard issues, only MP resolves 1 (sphinx-8713, 4.3%).

These results suggest that the information-preservation benefit of the Blackboard architecture is most apparent when the task falls within the agents' capability range.

TABLE 3. Resolve rate by difficulty level.

Difficulty	MP	BB	Hybrid
Easy (Django)	4/22 (18.2%)	7/22 (31.8%)	5/22 (22.7%)
Medium	1/5 (20.0%)	1/5 (20.0%)	1/5 (20.0%)
Hard	1/23 (4.3%)	0/23 (0.0%)	0/23 (0.0%)

2) Issue-Level Overlap

Five issues form a "common core" resolved by all three configurations. BB uniquely resolves two additional issues (django-12915, django-13028); MP uniquely resolves sphinx-8713. In total, 9/50 issues (18%) are resolved by at least one configuration.

B. INFORMATION FIDELITY SCORE

The IFS analysis provides clear evidence of the benefits of the Blackboard architecture (Table 4, Figure 3).

1) Coder Stage

BB achieves an average Coder IFS of 0.586 compared to 0.362 under MP, resulting in a 62% relative improvement

(Wilcoxon $W=36.5$, $p=0.058$, $n=28$ non-zero pairs; Figure 3a). As mentioned in Section III-B, the Coder's context under MP is $C^{MP}(\text{Coder}) = \{\text{out}(\text{Planner})\}$, whereas under BB it is $C^{BB}(\text{Coder}) = \{A, I\}$, which includes both the Planner's analysis and the original issue description I . Due to the absence of I from C^{MP} , the Coder is forced to rely entirely on the Planner's lossy summarization, which accounts for the 62% improvement in entity preservation.

2) Information Decay Pattern

For BB and Hybrid, IFS decays across the pipeline stages (Figure 3b): Planner (~ 0.74) $\rightarrow$ Coder (~ 0.56) $\rightarrow$ Reviewer (~ 0.48) $\rightarrow$ Tester (~ 0.08). This decline reflects a shift toward implementation tasks in downstream stages rather than reproducing terminology from the original issue description. A manual inspection of a sample of Tester outputs confirms that they primarily report pass/fail counts, the names of the FAIL_TO_PASS tests that were executed, and SWE-bench harness messages, while they do not generally reproduce the function or class identifiers from the original issue description; this is consistent with the low Tester IFS and indicates that Tester IFS is not a useful diagnostic for upstream information preservation.

3) IFS and Resolve Rate

While a higher IFS does not guarantee resolution, the two issues uniquely resolved under BB (django-12915 and django-13028) both exhibit substantially higher Coder IFS compared with MP. This pattern suggests that improved information preservation may have contributed to their successful resolution.

4) Per-Repository Breakdown

A concern that arises from the sample's repository composition is whether BB's IFS advantage might be driven by the over-representation of Django (22/50 issues). To address this, we recompute Coder-stage IFS separately for each repository in the sample (Table 5). The Coder-IFS advantage of BB over MP holds in *every* repository: from a modest +17% on Django (where MP already retains comparatively rich content) up to substantially larger gains on sympy (+178%), astropy (+187%), sphinx-doc (+79%), and pytest-dev (an effective 33 $\times$ improvement on issues where MP retained almost none of the original technical entities). The corresponding correct-file-targeting rates also rise under BB across every repository. The resolve-rate advantage, in contrast, is observed primarily on easy Django issues, consistent with the bottleneck model (Section VI-B): on the harder sympy and astropy issues the downstream patch-generation bottleneck dominates, so upstream gains do not translate into additional resolved issues. The IFS effect of the Blackboard architecture is therefore an architecture-level property of the communication layer rather than a Django-specific artifact.

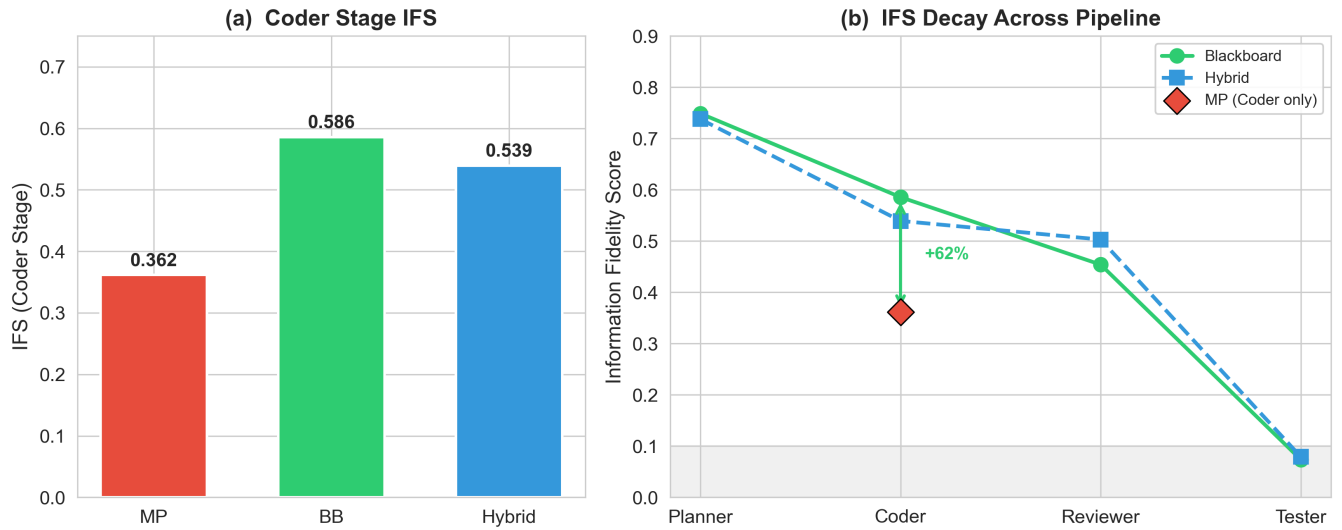


FIGURE 3. Information Fidelity Score (IFS) analysis. (a) Coder-stage IFS comparison between MP and BB. (b) IFS decay across pipeline stages for BB and Hybrid configurations.

TABLE 4. Information Fidelity Score by pipeline stage (mean $\pm$ std).

Comm.	Planner	Coder	Reviewer	Tester
MP	0.749 [†]	0.362 $\pm$ 0.385	N/A	N/A
BB	0.749 $\pm$ 0.308	0.586 $\pm$ 0.342	0.454 $\pm$ 0.361	0.074 $\pm$ 0.165
Hybrid	0.738 $\pm$ 0.332	0.539 $\pm$ 0.357	0.503 $\pm$ 0.340	0.079 $\pm$ 0.246

[†]Planner IFS for MP is identical to that for BB by design: the Planner is invoked with the same prompt and the same input (the original issue *I*) in all configurations, and the LLM is run with temperature 0 for deterministic generation. The intermediate text output of the MP Planner was not separately persisted; we therefore report the BB Planner-stage value, which is the value the MP Planner would produce in this experimental setting. Standard deviations are computed across the per-issue IFS values available for that stage.

TABLE 5. Per-repository Coder-stage IFS and correct file targeting (non-empty patches). Number in parentheses is the count of available IFS measurements.

Repo	N	Comm.	Resolved	Coder IFS	File targeting
django	22	MP	4/22	0.474	68.2%
		BB	7/22	0.555	76.2%
		Hybrid	5/22	0.541	72.7%
sympy	16	MP	0/16	0.207	20.0%
		BB	0/16	0.577	76.9%
		Hybrid	0/16	0.583	75.0%
sphinx-doc	3	MP	1/3	0.472	66.7%
		BB	0/3	0.847	100.0%
		Hybrid	0/3	0.875	100.0%
pytest-dev	2	MP	1/2	0.016	100.0%
		BB	1/2	0.532	100.0%
		Hybrid	1/2	0.032	100.0%
astropy	2	MP	0/2	0.147	50.0%
		BB	0/2	0.423	100.0%
pydata, pylint-dev, pallets	3	all	1/3	similar across modes	similar

5) Robustness of IFS to Matcher Choice

Because IFS relies on automated entity matching, we examine whether its results depend on the matcher implementation. We re-evaluated IFS on a 19-issue subset using a stricter fuzzy matcher based on Python's `difflib.SequenceMatcher` with a partial-match

threshold of 0.80. On this subset the rule-based matcher yields a BB-over-MP relative IFS gap of +39.3%; the fuzzy matcher yields +30.6%. Both matchers preserve the directional advantage of BB. The relative magnitude is somewhat sensitive to matcher strictness—as expected, because the rule-based matcher counts any case-insensitive substring hit while the

difflib matcher requires consecutive-token similarity above 0.80—but the qualitative finding that BB preserves more upstream information than MP is robust to this measurement choice.

C. FAILURE ANALYSIS

Patch formatting, rather than information quality, appears to be the primary bottleneck across all configurations (Table 6).

TABLE 6. Failure mode distribution.

Category	MP	BB	Hybrid
Resolved	6	8	6
Empty Patch	1	4	18
Patch Apply Error	43	38	26
Total	50	50	50

Patch Apply Error is the most common failure mode (MP: 43/50, BB: 38/50, Hybrid: 26/50), occurring when generated diffs contain context lines that do not match repository content.

Empty Patch failures are very frequent in Hybrid mode (18/50, 36%) compared with MP (1/50, 2%) and BB (4/50, 8%), suggesting that the dual-channel context overwhelms the Coder.

To characterize this failure mode concretely, we inspected each of the 18 Hybrid empty-patch runs. In every one of them, the Coder produced no parseable patch object at all: `blackboard_final_state.patches` is empty, even though the Reviewer was invoked three times as expected. This is not the same failure mode as a malformed or partially-correct diff; rather, the Coder appears to abandon the patch-emission step entirely when given the combined message-passing summary plus the full structured blackboard state. The same 18 issues are also unresolved under MP and BB, so the failures concentrate on intrinsically hard issues; Hybrid’s dual-channel context appears to suppress the Coder’s output on precisely the issues where the model already has the least useful signal. This corroborates the “context overload” hypothesis [28] with direct, in-experiment evidence and suggests that future hybrid designs should constrain the Coder’s input rather than offer both channels simultaneously.

The large gap between BB’s IFS advantage (62%) and its resolve rate advantage (4pp) is explained by this patch-formatting bottleneck; Section V-D provides a fine-grained decomposition.

D. PATCH QUALITY ANALYSIS

We break down the path from issue to resolution into intermediate stages (Table 7, Figure 4).

1) File Targeting

Among non-empty patches, the BB Coder correctly identifies the file(s) that require modification—closely related to software fault localization [29], [30]—in 82.6% of cases, compared with 55.1% for MP, corresponding to a 27.5

TABLE 7. Patch quality decomposition by communication mode ($N = 50$).

Metric	MP	BB	Hybrid
Non-empty patch rate	98.0%	92.0%	64.0%
Correct file targeting	55.1%	82.6%	78.1%
Resolve rate (of correct file)	22.2%	21.1%	24.0%

percentage-point improvement (Figure 2c). Hybrid achieves 78.1%, which is close to BB. This pattern directly mirrors the earlier IFS advantage, suggesting that shared-state access enables more accurate code localization.

2) Conditional Resolve Rate

Once the correct file is targeted, all three modes achieve similar resolve rates: MP 22.2%, BB 21.1%, Hybrid 24.0%. The near-identical conditional rates indicate that patch generation precision, rather than information quality, is the main constraint.

3) Reviewer Accept/Reject Dynamics

The Reviewer agent decides whether the Coder’s current patch should be accepted, returned for revision (rejected), or sent forward to the Tester at the iteration limit (3 by default). Aggregating Reviewer verdicts from `blackboard_final_state.reviews` across all 50 issues exposes a clear behavioral difference between BB and Hybrid. Under BB, the Reviewer issues 55 accept and 80 reject verdicts; first acceptance occurs on iteration 1 in 21 runs, on iteration 2 in 4, on iteration 3 in 8, and never in 17 runs. Under Hybrid, the corresponding numbers are 37 accept, 103 reject, with first acceptance on iteration 1 in only 13 runs and never-accepted in 29 runs. BB therefore both elicits more first-iteration acceptances and reduces the number of runs in which the Reviewer never accepts a patch, despite the loop being bounded at the same three iterations in all configurations. The MP runs do not persist explicit verdicts, but their distribution over iteration counts (5/1/44 for iters=1/2/3) is closer to Hybrid than to BB, again consistent with BB’s higher-quality patches being accepted earlier. These results indicate that BB’s information-preservation benefit propagates downstream into measurably more efficient Reviewer behavior, even though the resulting end-to-end resolve rate gain is modest because of the diff-generation bottleneck.

4) Information Flow Bottleneck Model

These findings support a stage-by-stage model of how information quality translates to task outcomes (Figure 4):

Issue $\rightarrow$ [IFS] $\rightarrow$ File Targeting $\rightarrow$ [Patch Gen.] $\rightarrow$ Patch Apply $\rightarrow$ [Test] $\rightarrow$ Resolve

BB outperforms MP in both IFS (+62%) and file targeting (+27.5pp), yet the three modes converge to roughly 22% conditional resolve rate at the patch generation stage. This bottleneck attenuates BB’s upstream advantages, explaining why a 62% IFS improvement yields only a 4pp gain in resolve rate.

Information Flow Bottleneck Model

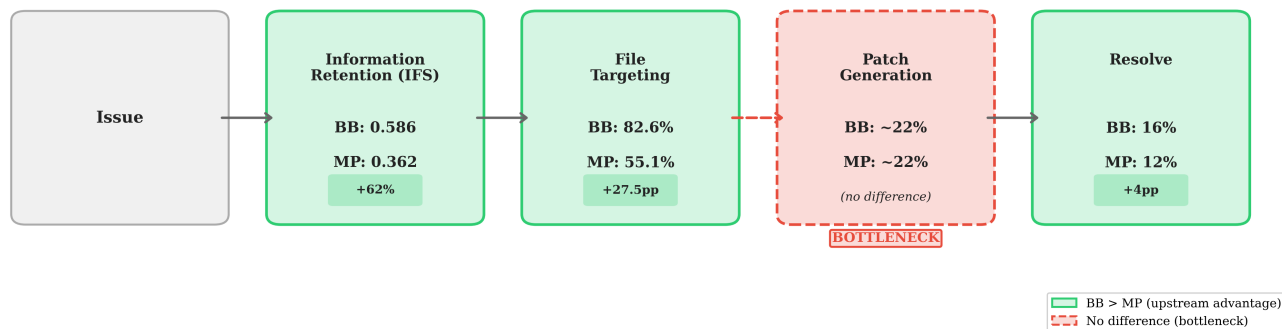


FIGURE 4. Information flow bottleneck model. BB outperforms MP in IFS and file targeting (upstream), but all configurations converge to approximately 22% conditional resolve rate at the patch generation stage (downstream).

E. TOOL USE ABLATION STUDY

To examine whether tool access can address the patch-generation bottleneck, we conducted an ablation study inspired by the Toolformer paradigm [31]. In this setup, the Coder is equipped with five tools: `read_file` (with optional line range), `search_code` (substring search across the working tree), `read_lines` (read a specified line range from a previously discovered file), `validate_patch` (dry-run apply via `git apply -check`), and `submit_patch` (final answer). The tools are exposed via a JSON schema, and the Coder is instructed to call `read_file` or `search_code` at least once before producing a diff, to use `validate_patch` prior to `submit_patch`, and to retry up to two times if validation fails before submitting anyway. The system prompt and tool descriptions are otherwise identical to the non-tool baseline; all other pipeline components (Planner, Reviewer, Tester) are unchanged. We evaluated 10 issues under both MP and BB (Table 8).

TABLE 8. Resolve rate with and without tool use (10-issue ablation).

Config	No Tool Use	Tool Use	Delta
MP	3/10	3/10	0
BB	4/10	4/10	0

Tool use does not improve resolve rates for either configuration. Although the `validate_patch` tool detects format errors, the LLM cannot correct them even after multiple retries. Token consumption increases 2–4x without yielding additional resolutions. These results confirm that the bottleneck is in the LLM’s intrinsic diff-generation ability, rather than information insufficiency or lack of tool access. We note that this ablation uses 10 issues per configuration, which is sufficient to rule out a large positive effect but not to establish a precise null; we therefore treat the result as directional evidence consistent with the bottleneck model rather than as

a definitive negative finding.

F. COST-EFFICIENCY

BB incurs significantly higher token costs: 54,849 tokens per issue versus MP’s 25,316 (2.17x, Wilcoxon $W = 34$, $p < 0.001$). Latency shows a similar pattern (171s vs. 106s, 1.61x, Wilcoxon $W = 63$, $p < 0.001$). To localize the overhead, we decompose token consumption per agent role (Table 9). The Coder consumes a comparable number of tokens in all three configurations ($\sim 17\text{--}20\text{k}$), and the Planner is also broadly comparable ($\sim 1.5\text{--}4.4\text{k}$). The dominant overhead under BB and Hybrid is concentrated at the Reviewer (28.2k for BB, 19.3k for Hybrid versus 3.5k for MP), because the Reviewer reads the full serialized blackboard state at each iteration. The Tester also incurs noticeably more tokens under BB and Hybrid (3.4k and 2.9k versus 0.6k under MP) for the same reason.

TABLE 9. Mean tokens consumed per agent role and per run, summed across iterations.

Comm.	Planner	Coder	Reviewer	Tester
MP	1,513	19,746	3,456	601
BB	4,428	18,895	28,168	3,359
Hybrid	3,944	17,151	19,306	2,877

When measured in tokens per resolved issue—arguably the more practically relevant cost figure than tokens per run—MP requires approximately 211,000 tokens, BB about 343,000 tokens, and Hybrid about 361,000 tokens. BB therefore spends roughly $1.6\times$ as many tokens per resolution as MP, even though it resolves $1.33\times$ as many issues. Practitioners considering BB at present resolve rates should regard this as a deliberate trade of fixed serialization overhead for higher upstream information quality, rather than as a free improvement. We also note that tokens consumed in successful runs are substantially lower than in failed runs in every configuration (MP: 13.4k vs. 26.9k; BB: 29.2k vs. 59.7k; Hybrid: 31.8k

vs. 44.8k), reflecting that successful runs typically terminate after one or two Coder–Reviewer iterations, while failures hit the iteration ceiling. The serialization overhead of BB would therefore be amortized to an increasingly favorable ratio if the downstream patch-generation bottleneck (Section VI-B) is addressed and the resolve rate rises.

G. CASE STUDY: DJANGO-13028

We examine django-13028, which is exclusively resolved by BB (Figure 5).

MP Failure: The Planner correctly identifies the root cause—a naming collision between a user-defined filterable field and Django’s internal `check_filterable` method. However, its summary compresses the diagnosis into approximately 679 tokens without exact traceback line numbers or method location (`Query.check_filterable` at line 1131 of `django/db/models/sql/query.py`). Instead, the Coder patches `Query.clone()` at line 355, which is a different method for managing the query cache. Despite three iterations, the Coder never locates the actual `check_filterable` method.

BB Success: The Coder reads the Planner’s analysis and the original issue description, which includes the full traceback: File “`django/db/models/sql/query.py`”, line 1131, in `check_filterable`. With direct access, the Coder generates a precisely targeted one-iteration patch adding a `hasattr(expression, ‘_meta’)` guard, correctly resolving the issue.

Concrete Impact: MP consumed 34,042 tokens across 3 iterations (131s) and failed; BB consumed 30,732 tokens in 1 iteration (67s) and succeeded.

Path-wise comparison. Tracing the agent interaction sequences makes the divergence explicit:

- **MP path:** Issue $\xrightarrow{C^{MP}(\text{Planner})=\{I\}}$ Planner produces a 679-token summary that drops the file/line traceback $\rightarrow$ Coder receives only this summary ($C^{MP}(\text{Coder}) = \{\text{out}(\text{Planner})\}$) and incorrectly patches `Query.clone()` (line 355) $\rightarrow$ Reviewer rejects $\rightarrow$ Coder re-attempts twice on a similarly wrong target $\rightarrow$ Tester fails (3 iterations).
- **BB path:** Issue $\xrightarrow{C^{BB}(\text{Planner})=\{I\}}$ Planner produces the same diagnosis $\rightarrow$ Coder receives the structured analysis and the original issue text including the traceback ($C^{BB}(\text{Coder}) = \{A, I\}$) $\rightarrow$ Coder targets `Query.check_filterable` on the first try $\rightarrow$ Reviewer accepts $\rightarrow$ Tester passes (1 iteration).

The divergence between the two paths occurs precisely at the Planner $\rightarrow$ Coder handoff, where MP’s lossy natural-language compression discards the file/line information that survives intact in BB’s shared state. This example illustrates how message passing can introduce information loss during summarization: even when the Planner’s diagnosis is correct, the compressed representation may omit the precise location details required for accurate patch generation.

H. STATISTICAL SUMMARY

Table 10 consolidates the statistical test results across all comparisons.

TABLE 10. Statistical test results. * indicates significance at $\alpha = 0.05$.

Test	Comparison	Statistic	p-value
McNemar	MP vs BB	$b = 1, c = 3$	$p = 0.625$
McNemar	MP vs Hybrid	$b = 1, c = 1$	$p = 1.000$
McNemar	BB vs Hybrid	$b = 2, c = 0$	$p = 0.500$
Cohen’s h	BB vs MP	$h = 0.116$	negligible
Odds Ratio	BB vs MP	OR=1.40	CI [0.45, 4.37]
Wilcoxon	Tokens: MP vs BB	$W = 34$	$p < 0.001^*$
Wilcoxon	Tokens: MP vs Hy	$W = 183$	$p < 0.001^*$
Wilcoxon	Latency: MP vs BB	$W = 63$	$p < 0.001^*$
Wilcoxon	IFS: MP vs BB	$W = 36.5$	$p = 0.058$

VI. DISCUSSION

A. THE INFORMATION PRESERVATION ADVANTAGE

Our results indicate that the Blackboard architecture improves information preservation in multi-agent SE pipelines. The 62% increase in Coder-stage IFS (0.586 vs. 0.362) suggests that shared-state access helps retain key technical entities more effectively. This observation is consistent with previous work on Blackboard architectures for reasoning systems [10] and data science workflows [11], extending these findings to software engineering.

The mechanism behind this improvement follows from the context functions defined in Section III-B. Under MP, $I \notin C^{MP}(\text{Coder})$: the Coder’s only source of information about the issue is the Planner’s natural language summary out(Planner), which compresses and paraphrases the original content. Under BB, $I \in C^{BB}(\text{Coder})$: the Coder has access to both the Planner’s structured analysis A and the original issue text I , allowing it to cross-reference and recover details that might be absent from the Planner’s summary. This architectural guarantee of information access is the fundamental advantage of the Blackboard architecture. The marginal significance ($p = 0.058$) with 28 non-zero paired differences suggests that a modestly larger experiment (~ 35 – 40 pairs) could yield $p < 0.05$.

We argue that **IFS and file targeting—not resolve rate—constitute the primary empirical contributions** of this study. Resolve rate is a coarse binary outcome that can be influenced by downstream bottlenecks beyond the communication layer’s control. IFS and file targeting isolate the pipeline stages that the communication architecture directly affects, providing a more precise and actionable signal. The 27.5 percentage-point file targeting advantage (82.6% vs. 55.1%) is a large and practically meaningful effect that would benefit downstream patch generation methods.

B. WHY IS THE RESOLVE RATE DIFFERENCE LIMITED?

A central question is the gap between the large IFS improvement (62%) and the modest resolve rate gain (4pp). The patch quality analysis (Section V-D) together with the tool

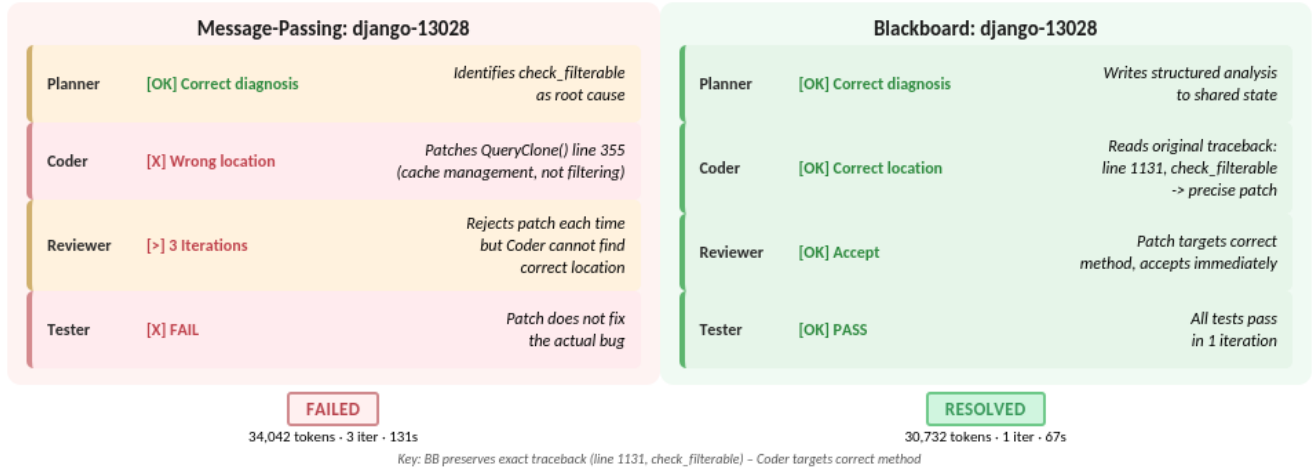


FIGURE 5. Case study: `django-13028`. Under MP, the Planner’s compressed summary omits precise location details, causing the Coder to patch the wrong method. Under BB, direct access to the original issue traceback enables a correctly targeted one-iteration fix.

use ablation (Section V-E) reveal a consistent causal chain, which we formalize as an *information flow bottleneck model* (Figure 4):

- 1) **Upstream: BB significantly outperforms MP.** BB achieves a 62% higher IFS, corresponding to a 27.5pp advantage in correct file targeting (82.6% vs. 55.1%).
- 2) **Downstream: A systematic bottleneck limits all modes.** Once the correct file is identified, all modes achieve $\sim 22\%$ conditional resolve rate. We attribute this convergence to the LLM’s difficulty in reliably generating context-correct unified diffs—a property we observe empirically across all 50 issues and all three configurations rather than infer from prior work: the $\sim 78\%$ of correct-file patches that nevertheless fail to apply or to pass tests do so primarily because of mismatched context lines, incorrect line numbers in hunk headers, or off-by-one offsets in the generated diffs (see also Section V-E, where giving the Coder direct file-read and validate tools does not improve this rate).
- 3) **Tool use does not resolve the bottleneck.** Despite file access and `validate_patch` capability, the LLM cannot generate corrected patches (Section V-E). Token consumption increases 2–4x without improving resolve rates.
- 4) **The bottleneck attenuates upstream advantages.** BB’s 27.5pp file-targeting advantage translates to approximately $27.5 \times 0.22 \approx 6$ additional resolve-rate percentage points in expectation—close to the observed 4pp gain.

These findings suggest that BB’s observed advantage closely matches what would be expected under the downstream bottleneck. As LLM diff-generation capabilities improve, or alternative patch representations bypass this constraint, the upstream advantages of BB may translate more fully into improvements in resolve rate.

The predictive accuracy of this model deserves emphasis.

From BB’s 27.5pp file-targeting advantage and the $\sim 22\%$ conditional resolve rate, the model predicts an expected improvement in resolve rate of $27.5 \times 0.22 \approx 6$ pp. The observed gain is 4pp (16% vs. 12%), falling within the expected range given sampling variance on 50 issues. This correspondence between predicted and observed effects provides strong internal validity for the bottleneck model, regardless of whether the difference in resolve rates itself is statistically significant.

A note on statistical power. A post-hoc power analysis indicates that detecting a 4 percentage-point difference (12% vs. 16%) using McNemar’s test at $\alpha = 0.05$ with 80% power would require approximately $N = 300$ paired observations, roughly six times the size of our sample. Because most pipeline runs share a downstream bottleneck (Section V-D), discordant pairs between MP and BB are intrinsically scarce ($b + c = 4$ in our experiment), which compounds the power problem on this dataset: even a doubled sample at the present discordance rate would still leave the binary outcome underpowered. Accordingly, this study was not designed to establish statistical significance for resolve rate alone, and the 4pp resolve-rate improvement should be interpreted as descriptive rather than confirmatory. Resolve rate is interpreted as one component of a multi-metric evaluation strategy, where IFS ($p = 0.058$), file targeting (+27.5pp), the bottleneck model, the tool-use ablation, and the case study together support a consistent explanation. We view this convergence of evidence as more informative than any single p-value.

C. WHY DOES HYBRID NOT OUTPERFORM?

The Hybrid configuration (12.0%) matches MP rather than exceeding BB. This pattern can be explained by the context functions defined in Section III-B: C^{Hy} is a strict superset of both C^{MP} and C^{BB} , combining the predecessor’s natural-language output with the full shared state. When presented with this larger context—consistent with prior findings that LLMs struggle to make effective use of very long inputs [28]—the Coder often produces no diff output. Specifically,

36% of Hybrid runs generate empty patches (18/50), compared with 2% for MP and 8% for BB.

However, among non-empty patches, Hybrid achieves a file-targeting rate of 78.1%, close to BB's 82.6%, and its conditional resolve rate (24.0%) is the highest among the three configurations. These results suggest that the limitation arises from input format and context size, rather than insufficient information quality. Therefore, future hybrid designs may benefit from mechanisms that limit context overload, such as selective sharing, progressive disclosure, or memory-management techniques in which the relevant subset of the shared state is surfaced to the Coder on demand rather than as a single combined context [32].

D. IMPLICATIONS FOR PRACTICE AND SYSTEM DESIGN

Adopt shared state for long pipelines. The information-preservation advantage of BB is likely to increase as pipeline length grows, because each MP handoff introduces an additional compression step. For pipelines with two or more stages, a shared-state design may therefore be a sensible default architectural choice.

Communication and code generation are orthogonal concerns. Communication architecture optimization (upstream: information preservation, file targeting) and code generation optimization (downstream: patch correctness) address different stages of the pipeline. Systems should adopt Blackboard for upstream quality while investing separately in downstream improvements—such as alternative patch representations, retrieval-augmented generation [33] for context lines, or model fine-tuning for diff generation. The combination is expected to be multiplicative.

IFS as a diagnostic metric. IFS provides stage-by-stage diagnostics that are independent of task success. Despite similar resolve rates, IFS differences reveal information-quality gaps that may become significant as other bottlenecks are addressed.

Avoid naive hybrid designs. Combining communication channels without careful integration may reduce performance, as illustrated by the 36% empty-patch rate observed in the Hybrid configuration.

E. BEYOND RESOLVE RATE: A CASE FOR MULTI-METRIC EVALUATION

A reviewer might ask: if resolve rate is not statistically significant, what has this study demonstrated? We argue that evaluation in multi-agent SE systems should not rely solely on end-to-end binary outcomes. Resolve rate collapses the entire pipeline into a single binary indicator, making it insensitive to improvements at individual stages when failures in other stages dominate the overall outcome. In our experiments, 86–88% of runs fail during patch application regardless of communication mode, creating a noise floor that obscures upstream improvements.

Our multi-metric evaluation decomposes the pipeline into measurable stages. IFS captures information preservation, file

targeting measures code localization accuracy, and conditional resolve rate reflects patch generation precision. This decomposition shows that BB substantially outperforms MP on the stages it is intended to affect (IFS: +62%, file targeting: +27.5pp), while the stage it does not influence—patch generation—remains limited, with a roughly 22% conditional resolve rate across all configurations. Without this decomposition, one might conclude that communication architecture has little effect, when in fact it improves the stages it directly influences while a downstream bottleneck constrains the final outcome.

These findings suggest that future evaluations of multi-agent SE systems would benefit from similar decomposition strategies that report intermediate metrics alongside end-to-end results. Such reporting enables meaningful comparisons even when sample sizes are insufficient to detect statistically significant differences in final outcomes. It also provides clearer guidance about which stages of the pipeline require improvement.

VII. THREATS TO VALIDITY

Internal Validity. We use a single LLM (Claude Sonnet 4.5) with temperature set to 0. This design choice removes model-related variance as a potential confounding factor, ensuring that observed differences across configurations can be attributed to the communication architecture rather than to variation in model behavior. The trade-off is reduced generalizability: the exact magnitude of the observed improvements (e.g., IFS gains or differences in resolve rates) may vary across models [1]. We hypothesize that the underlying direction of BB's advantage—improved information preservation through shared-state access—is an architectural property that should generalize across LLMs, because shared-state access mechanically guarantees verbatim availability of the original issue text regardless of model. This hypothesis, however, remains to be tested directly with other models and should not be treated as an empirical finding of the present study. The sequential pipeline also introduces an ordering effect, where a weak Planner analysis can influence all downstream agents regardless of communication mode. To reduce this effect, we use identical Planner prompts across all configurations.

External Validity. Our sample of 50 issues from SWE-bench Lite is limited to well-maintained Python repositories. Django issues are overrepresented (22/50), and because BB's resolve-rate advantage is concentrated in easy Django issues, the observed effect may partly reflect repository-specific factors, such as Django's detailed issue descriptions and well-structured codebase. As a result, the findings may not generalize to other programming languages, less-documented repositories, or software engineering tasks beyond bug fixing. However, the IFS advantage appears consistently across all repositories in our sample, not only Django, suggesting that the underlying information-preservation mechanism is not tied to a particular repository.

Construct Validity. IFS relies on rule-based entity extraction, which may fail to capture semantically equivalent

references or may count incidental mentions as evidence of preservation. As discussed in Section VI-B, resolve rate alone may underestimate the effect of communication architecture. The intermediate metrics used in this study provide a more complete assessment.

Statistical Validity. With 50 issues and only 4 discordant pairs, McNemar’s test has limited statistical power to detect the observed 4 percentage-point difference in resolve rate. A post-hoc power analysis indicates that $N \approx 300$ paired observations would be required to detect a difference of this magnitude with 80% power at $\alpha = 0.05$. The IFS result ($p = 0.058$) is close to, but does not reach, the conventional significance threshold. To mitigate the risk of overinterpreting p-values under limited sample size, we report effect sizes and confidence intervals alongside hypothesis tests. These statistics provide a more stable characterization of the observed differences and facilitate comparison with future studies.

VIII. CONCLUSION AND FUTURE WORK

A. CONCLUSION

This paper introduced SE-Blackboard, applying the Blackboard shared-state architecture to multi-agent software engineering pipelines. Through controlled experiments on 50 SWE-bench Lite issues, we show that communication architecture measurably affects information preservation and intermediate task performance. Our main findings are:

- 1) **Information Preservation:** BB improves Coder-stage IFS by 62% compared with MP (0.586 vs. 0.362, $p = 0.058$), indicating that shared-state access substantially reduces information loss across pipeline stages.
- 2) **Code Localization:** BB’s information advantage translates into 82.6% correct file targeting versus 55.1% for MP, improving the key prerequisite for successful patch generation.
- 3) **Information Flow Bottleneck Model:** Once the correct file is identified, all configurations converge to a $\sim 22\%$ conditional resolve rate, indicating that LLM diff-generation reliability currently limits end-to-end performance. A tool-use ablation further shows that this bottleneck cannot be removed through tool access alone.
- 4) **Resolve Rate:** BB achieves a 16.0% resolve rate compared with 12.0% for MP (a 33% relative improvement), consistent with the bottleneck model’s prediction that a 27.5pp file-targeting advantage should yield roughly 6pp expected improvement in resolve rate.
- 5) **Hybrid Caution:** Directly combining communication channels can degrade performance due to context overload, producing a 36% empty-patch rate, although successful Hybrid patches achieve file-targeting accuracy close to BB.

These results highlight communication architecture as an important design dimension for multi-agent SE systems. The proposed information flow bottleneck model further provides a conceptual framework for understanding how improve-

ments in upstream information quality translate into downstream task outcomes.

B. FUTURE WORK

Alternative Patch Representations. The unified diff bottleneck is not resolved through tool augmentation alone. Future work could explore patch representations that avoid strict context-line matching. Three concrete directions are most promising: (i) *search-and-replace specifications*, where the Coder emits a list of (`old_block`, `new_block`) pairs and a deterministic tool performs the substitution—this avoids context-line matching entirely while keeping outputs short; (ii) *AST-level edits*, where the Coder produces structured tree transformations applied by a language-aware engine—this guarantees syntactic validity but requires per-language tooling; and (iii) *whole-file rewriting*, where the Coder emits the full content of the modified file and the diff is recovered by external comparison—this scales poorly to large files but eliminates the diff-generation step at the model level.

Improved Diff Generation. Improving diff generation itself is another promising direction. Potential approaches include fine-tuning on patch corpora, constrained decoding for diff syntax, deterministic post-hoc repair of corrupt hunk headers, or retrieval-augmented generation [33] to retrieve accurate context lines while retaining the unified diff format. A practical compromise—used outside this study—is to apply patches with progressively more lenient apply heuristics (`git apply → patch -F0 → patch -F3`) so that minor context errors do not cause outright failure; this trades a small risk of incorrect application against substantially higher apply rates.

Debate Topology. The SE-Blackboard framework also supports a debate-style topology in which multiple Coders generate competing patches and a Reviewer selects the best candidate. Prior work shows that multi-agent debate can improve factuality and reasoning in LLMs [34]. Evaluating this topology across different communication architectures would help determine whether the benefits of shared state compound as agent diversity increases.

Larger-Scale Evaluation. Future studies should extend the evaluation to the full SWE-bench Lite dataset (300 issues) and include multiple LLMs. Such experiments would provide sufficient statistical power for resolve rate comparisons and allow stronger conclusions about generalizability.

Broader Applications. The Blackboard architecture may benefit other software engineering tasks, such as feature development, code review, and test generation, as well as domains that require precise information transfer in multi-agent systems.

DATA AND CODE AVAILABILITY

The source code for the SE-Blackboard framework is publicly available on GitHub at <https://github.com/el4435/SE-Blackboard> and archived as an executable capsule on Code Ocean. The experiment data, including all result files,

statistical analyses, and figures, are available on Zenodo at <https://doi.org/10.5281/zenodo.18911615>.

ACKNOWLEDGMENT

Large language models were used during the preparation of this work. ChatGPT was used to assist with proofreading and language editing of the manuscript. Claude (Anthropic) was used to assist with portions of the implementation, specifically scaffolding the agent base classes (Planner, Coder, Reviewer, Tester), drafting the SWE-bench Docker harness wrapper, and producing initial versions of the analysis scripts used in Section V. All generated code was reviewed by the first author, integrated into the project's regression test suite (63 unit tests), and validated against the experimental results reported in this paper. The experiment design, all empirical claims, the statistical analyses, and the conclusions were produced and verified by the authors, who take full responsibility for the work.

REFERENCES

- [1] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [2] S. Hong et al., "Metagpt: Meta programming for a multi-agent collaborative framework," in *Proceedings of ICLR*, 2024.
- [3] C. Qian et al., "Chatdev: Communicative agents for software development," in *Proceedings of ACL*, 2024.
- [4] J. Yang et al., "Swe-agent: Agent-computer interfaces enable automated software engineering," *arXiv preprint arXiv:2405.15793*, 2024.
- [5] D. Huang et al., "Agentcoder: Multi-agent-based code generation with iterative testing and optimisation," *arXiv preprint arXiv:2312.13010*, 2024.
- [6] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman et al., "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [7] M. Cemri, M. Z. Pan, S. Yang et al., "Why do multi-agent llm systems fail?" in *Proceedings of NeurIPS Datasets and Benchmarks Track*, 2025.
- [8] X. Fang et al., "Rtadev: Intention aligned multi-agent framework for software development," in *Findings of ACL*, 2025.
- [9] H. P. Nii, "Blackboard systems: The blackboard model of problem solving and the evolution of blackboard architectures," *AI Magazine*, vol. 7, no. 2, pp. 38–53, 1986.
- [10] Z. Wang et al., "Lbmas: Exploring llm-based multi-agent blackboard systems for reasoning tasks," *arXiv preprint arXiv:2502.11051*, 2025.
- [11] A. Salemi, M. Parmar, P. Goyal, Y. Song, J. Yoon, H. Zamani, H. Palangi, and T. Pfister, "Llm-based multi-agent blackboard system for information discovery in data science," *arXiv preprint arXiv:2510.01285*, 2025.
- [12] B. Rozière et al., "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.
- [13] Q. Wu et al., "Autogen: Enabling next-gen llm applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.
- [14] Y. Zhang et al., "Autocoderover: Autonomous program improvement," *arXiv preprint arXiv:2404.05427*, 2024.
- [15] Y. Chen et al., "Coder: Issue resolving with multi-agent and task graphs," *arXiv preprint arXiv:2406.01304*, 2024.
- [16] C. S. Xia et al., "Agentless: Demystifying llm-based software engineering agents," *arXiv preprint arXiv:2407.01489*, 2024.
- [17] W. Tao, Y. Zhou, W. Zhang, and Y. Wang, "Magis: Llm-based multi-agent framework for github issue resolution," *arXiv preprint arXiv:2403.17927*, 2024.
- [18] J. He, C. Treude, and D. Lo, "Llm-based multi-agent systems for software engineering: Literature review, vision and the road ahead," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, 2025.
- [19] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [20] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *Proceedings of ICLR*, 2023.
- [21] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [22] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of UIST*, 2023.
- [23] L. D. Erman, F. Hayes-Roth, V. R. Lesser, and D. R. Reddy, "The hearsay-ii speech-understanding system: Integrating knowledge to resolve uncertainty," *ACM Computing Surveys*, vol. 12, no. 2, pp. 213–253, 1980.
- [24] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths, "Cognitive architectures for language agents," *Transactions on Machine Learning Research*, 2024.
- [25] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "Swe-bench: Can language models resolve real-world github issues?" in *Proceedings of ICLR*, 2024.
- [26] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
- [27] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [28] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.
- [29] W. E. Wong, R. Gao, Y. Li, R. Abreu, and F. Wotawa, "A survey on software fault localization," *IEEE Transactions on Software Engineering*, vol. 42, no. 8, pp. 707–740, 2016.
- [30] Y. Lou, B. Chen, L. Zhang, and D. Hao, "Boosting coverage-based fault localization via graph-based representation of test failures," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2021, pp. 800–811.
- [31] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [32] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang, "Memorybank: Enhancing large language models with long-term memory," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 17, 2024, pp. 19724–19731.
- [33] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel et al., "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [34] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving factuality and reasoning in language models through multiagent debate," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.



ERXI LIU received the B.S. degree in computer science from Syracuse University, Syracuse, NY, USA, in 2022. He is currently pursuing the M.S. degree in computer engineering at the Tandon School of Engineering, New York University, New York, NY, USA. He was previously with ZTE Corporation, where he worked on intelligent operations and maintenance (AIOps). He was also a Research Intern with the Chinese Academy of Sciences, focusing on operating system migration for robotic frameworks, and with Zhejiang Dajingyi Lean Co., Ltd., where he developed AI-based algorithms for industrial pipeline inspection. His experience spans both industry and research across software systems and applied AI. His research interests include multi-agent systems, large language models, software engineering automation, AI-driven DevOps, and human-AI collaboration.



QIANG ZHU received the Ph.D. degree in optical engineering from the University of Chinese Academy of Sciences, Beijing, China, in 2022. He is currently an Associate Researcher and Master's Supervisor at the National Institute of Metrology, China, where he serves as the Head of the Optical Rotation Metrology Laboratory. He is a registered expert and member of ISO/TC 130 and a member of the National Printing Machinery Standardization Technical Committee. He also serves as a Master's Supervisor at Beijing Institute of Graphic Communication and Xi'an University of Technology. He has participated in six national key projects, including the National Key R&D Program, and has led the drafting of two national standards. He has received one first prize and two third prizes at the provincial and ministerial level for scientific and technological achievements. He has published over 20 academic papers, with more than three indexed by EI. His research interests include photoelectric precision instruments, photoelectric detection technologies, and optical rotation metrology.



XINRU DONG received the B.S. degree from Mudanjiang Normal University, Mudanjiang, China, in 2023. She is currently pursuing the M.S. degree at the School of Mechanical and Electrical Engineering, Beijing Institute of Graphic Communication, Beijing, China. She is also a Visiting Student with the Division of Dimensional Metrology, National Institute of Metrology, China, where she participates in research on precision measurement and intelligent instrumentation. Her research interests include dimensional metrology, AI-assisted measurement systems, multi-agent systems, and the application of large language models to engineering workflows.

...